



COMP1850 Programming

School of Computer Science, University of Leeds

Worksheet 1.2

This worksheet contains a combination of formative activities and activities that contribute towards your portfolio.

- Portfolio activities are indicated in the title of the task.
- Activities marked by (*) are advanced activities and may take more time to complete.

Expectations:

1. **Timeliness:** You should complete all the tasks in the order provided and submit your portfolio task to Gradescope before the deadline.
2. **Presentation:** You should present all your work clearly and concisely following any additional guidance provided by the module staff in the module handbook.
3. **Integrity:** You are responsible that the evidence you submit as part of your portfolio evidence is **entirely your own work**. You can find out more about Academic integrity on the Skills@library website. All work you submit for assessment is subject to the academic integrity policy.

Feedback: Feedback on formative activities will be provided via Lab classes and tutorials. Feedback on evidence submitted as part of the portfolio will be available on Gradescope.

Support opportunities: Support with the activity sheet is available in the Lab classes and tutorials. Individual support is available via office hours.

Expected complete date: Friday 16th October

Portfolio tasks should be entirely your own work. AI tools (such as Microsoft Copilot) can be used in an assistive role. You are permitted to use AI tools for specific defined processes within the assessment.

Within this assessment you **may** use Generative AI to:

- help you start to understand more complex ideas by providing accessible summaries.
- test your knowledge against a piece of content.
- help identify and correct spelling mistakes and grammatical errors in your work.
- provide feedback and advice on your overall coding style.

You must **not** use Generative AI to:

- produce written content (code, comments or any other content) which you then submit as your work, regardless of whether you make changes to it.
- rewrite or make changes to any of your work.

Learning Outcomes:

- Develop programs that use the fundamental programming constructs: assignment and expressions, basic I/O, conditional statements
- Develop programs that effectively use the different structured data types provided in the language like lists, dictionaries, and sets

Portfolio Task

Task 1

Edit the file `task1.py`. In this file, write a program that

- Asks the user to enter an integer grade in the range 0 to 100
- Converts that grade into a result of Distinction, Pass or Fail, where
 - Fail = 0 – 39
 - Pass = 40 – 69
 - Distinction = 70 – 100
- Prints out the numeric grade and the result, in the same format as the examples below:
82 is a Distinction
57 is a Pass
36 is a Fail

If the user enters something that isn't a number, or they enter an integer value outside the required range, the program should immediately exit, after first displaying this exact error message on the standard error channel:

```
Error: Grade must be an integer between 0 and 100
```

Use the `exit()` function from Python's `sys` module to achieve this. Here is an example of how you import this module and use the `exit()` function:

```
import sys  
sys.exit("Error!")
```

Hint: strings have a method `isdecimal()` that will tell you whether the string consists of characters that could represent a decimal integer.

Task 2

Edit the file `task2.py`. In this file, write a program that

- Prompts the user to input a sequence of float values
- Reads these values into a list, using code that we have provided
- Finds the minimum, maximum, mean and median of these values
- Prints each statistic on a separate line

For example, given the list

```
[4.5, 3.0, 1.2, 7.0, 6.3]
```

your program should print the following lines as its output:

```
Minimum = 1.2
Maximum = 7.0
Mean = 4.4
Median = 4.5
```

If no numbers were entered, the program should terminate with a non-zero exit status and display the following error on the standard error channel:

```
Error: no numbers provided
```

Use the `sys.exit()` function to do this, as you did in `task1.py`.

Notes

Read the numbers from the user using the function that we have provided for this purpose. The function is defined in the file `util.py`.

You can use it like this:

```
from util import read_numbers
numbers = read_numbers()
```

Your program must NOT use any Python modules other than the `sys` and `util` modules mentioned above.

Your program should perform its calculations without using loops.

Hints

Minimum, maximum and mean can be determined in a fairly straightforward manner, using four of Python's [built-in functions](#).

The median is the 'middle' number, after arranging numbers in ascending order. You can use [one of Python's list methods](#) to help you arrange the numbers in this way.

Note that there won't be a single middle number when the list size is even. Your code will need to handle this appropriately. See the [definition of median](#) for further information on this.

*Extension Task

This is optional and covers more advanced features of Python that will be covered in later weeks of the module. Try doing this if you already have some experience of Python and you finished the portfolio tasks quickly.

Copy `task2.py` to a new file. Improve this copy of the program by modifying it so that it reads numbers from a file instead of prompting the user to enter them interactively. Write a `read_file` function in `util.py` that performs this part of the task. It should take the filename as a parameter.

Modify the program so that the filename is supplied on the command line. So if the program is named `stats.py` and you have a file named `numbers.txt`, the program is run like this:

```
python stats.py numbers.txt
```

Finally, add some code that computes the **standard deviation** of the numbers in the list. Implement this by writing a function named `standard_deviation`, which takes the list of numbers as a parameter.

Sample Exam Question

Consider the following fragment of Python code:

```
data = [0.7, 1.2, 0.9, 1.0]
print(data[index])
```

- (a) What happens when this code runs, if `index` has the value 1?
- (b) What happens when this code runs, if `index` has the value 4?
- (c) What happens when this code runs, if `index` has the value `-1`?